

Curso de Go

Aula 2 - Estrutura de um Programa em Go

Professor : Antônio Marcelo

NÚCLEA
ACADEMY

 `código_`^[3]
BRAZUCA





Nessa Aula nos Vamos Ver

- **Como um programa Go é organizado**
- **Entender main, pacotes e init**
- **Criar a base para qualquer projeto em Go**
- **Criar um programa na parte prática**



Visão Geral da Estrutura

- **Todo programa Go segue uma estrutura simples e padronizada**
- **Ponto de entrada definido claramente**
- **Organização baseada em pacotes**

Exemplo de código

```
package main
```

```
import "fmt"
```

```
func main() {  
    fmt.Println("Olá, mundo!")  
}
```



O que é o package

- **Todo arquivo Go começa com a definição de um pacote**
- **Pacotes organizam o código em módulos reutilizáveis**
- **O pacote principal é sempre chamado main**

Exemplo de código

```
package main
```



O papel da função main

- A função main é o ponto de entrada do programa
- É onde a execução começa
- Só existe em programas executáveis (package main)

Exemplo de código

```
func main() {  
    fmt.Println("Executando  
programa")  
}
```



O que são imports

- Imports permitem usar código de outros pacotes
- Podem ser da biblioteca padrão ou externos
- Sempre declarados no início do arquivo

Exemplo de código

```
import "fmt"
```



Usando múltiplos imports

- Podemos importar vários pacotes
- Forma recomendada usa bloco

Exemplo de código

```
import (  
    "fmt"  
    "math"  
)
```

Uso:

```
fmt.Println(math.Sqrt(16))
```



Organização do fluxo de execução

Ordem de execução do Go:

1. Inicialização de variáveis globais
2. Execução de funções `init()`
3. Execução da função `main()`

Exemplo de código

```
package main
```

```
import "fmt"
```

```
var nome = "Antonio"
```

```
func init() {  
    fmt.Println("Init executado")  
}
```

```
func main() {  
    fmt.Println("Main executado")  
    fmt.Println("Nome:", nome)  
}
```



O que é a função init

- **Função especial executada automaticamente**
- **Não recebe parâmetros e não retorna valores**
- **Pode existir mais de uma por pacote**

Exemplo de código

```
func init() {  
    fmt.Println("Inicializando o  
programa...")  
}
```



Exemplo Init+main

```
import "fmt"

var nome string = "Antonio"

func init() {
    fmt.Println("Init executado")
}

func main() {
    fmt.Println("Main executado")
    fmt.Println("Nome:", nome)
}
```

Saída Esperada
Init executado
Main executado
Nome: Antonio



Boas práticas

- **Use init apenas para inicializações necessárias**
- **Evite lógica complexa dentro do init**
- **Mantenha main simples e clara**
- **Separe código em pacotes conforme o projeto cresce**

```
1 package main
2 import "fmt"
3 type rect struct {
4     width, height int
5 }
6 Este método area possui um tipo receptor de *rect.
7
8 func (r *rect) area() int {
9     return r.width * r.height
10 }
11 Métodos podem ser definidos para qualquer tipo de receptor ponteiro ou valor. Aqui temos um exemplo de receptor
12
13 func (r rect) perim() int {
14     return 2*r.width + 2*r.height
15 }
16 func main() {
17     r := rect{width: 10, height: 5}
18     Nós podemos chamar 2 métodos definidos na nossa estrutura.
19
20     fmt.Println("area: ", r.area())
21     fmt.Println("perim:", r.perim())
22     Go trata automaticamente conversões entre valores e ponteiros para métodos de chamada. Você pode querer usar um
23
24     rp := &r
25     fmt.Println("area: ", rp.area())
26     fmt.Println("perim:", rp.perim())
27 }
```

Parte Prática

Fixar estrutura de um programa Go